МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

ЗВІТ

з лабораторної роботи № 5
«Виявлення та усунення вразливостей за OWASP Top 10»
з дисципліни «Безпека програмного забезпечення»

Варіант 14

Виконав:
здобувач групи ІП-34
Скачков І. О.

Перевірив:
Курченко О. А.

Київ — 2026

1 МЕТА ТА ЗАВДАННЯ РОБОТИ

Мета роботи — набути практичних навичок повного циклу роботи з дефектом захищеності вебзастосунку: від підтвердження експлуатації вразливості в ізольованому середовищі до її усунення в коді та закріплення результату регресійним тестом.

Завдання роботи:

1) розгорнути ізольований стенд із вразливим застосунком засобами Docker Compose, доступний лише на 127.0.0.1;
2) визначити п’ять вразливостей варіанта в коді за маркерами ВРАЗЛИВІСТЬ N;
3) підтвердити експлуатацію кожної у формі доказу варіанта (знімок із curl-командою);
4) усунути дефект мінімальною зміною коду, зберігши штатну поведінку маршруту;
5) розробити регресійний тест для кожної вразливості (червоний на вразливому коді, зелений на виправленому);
6) повторно перевірити й обґрунтувати усунення всіх п’яти вразливостей.

Параметри варіанта отримано командою `python -m common.variants 14 --lab 5`:

```
  Варіант 14

    LAB5
      required:
        - IDOR у перегляді документів
        - відображений XSS у повідомленні про помилку
        - відсутність перевірки прав в адміністративному маршруті
        - SQL-ін'єкція у пошуку
        - CSRF у зміні профілю
      proof_form: знімок екрана з відповіддю сервера та curl-командою
```

Таблиця 1.1 – П’ять вразливостей варіанта 14

Код	Клас вразливості	CWE / OWASP	Маршрут	Маркер у app.py
B1	SQL-ін’єкція у формі входу	CWE-89 / A03	/login	ВРАЗЛИВІСТЬ 2
B3	Відображений XSS у пошуку	CWE-79 / A03	/search	ВРАЗЛИВІСТЬ 6
B4	CSRF у зміні профілю	CWE-352 / A01	/profile	ВРАЗЛИВІСТЬ 8
B5	IDOR у перегляді документів	CWE-639 / A01	/document/<id>	ВРАЗЛИВІСТЬ 5

Код	Клас вразливості	CWE / OWASP	Маршрут	Маркер у app.py
B10	Відсутність перевірки прав	CWE-862 / A01	/admin/users	ВРАЗЛИВІСТЬ 12

Форма доказу варіанта — Д1 (знімок екрана з curl-командою та відповіддю сервера). Формулювання варіанта називає клас узагальнено, тому орієнтир — номер CWE й маркер: єдина SQL-ін'єкція (B1) розміщена у /login, а відображений XSS (B3) — у /search.

2 РОЗГОРТАННЯ СТЕНДА

Стенд із двох служб спільної ізольованої мережі `bpz-isolated` розгорнуто командою `docker compose up -d --build`. Служба `vulnapp` публікує застосунок лише на петльовому інтерфейсі `127.0.0.1:8080`, служба `internal` портів назовні не має (рис. 2.1). Стан служб:

```
NAME            STATUS              PORTS
bpz-internal    Up 8 minutes        5000-5001/tcp
bpz-vulnapp     Up About a minute   127.0.0.1:8080->5000/tcp
```

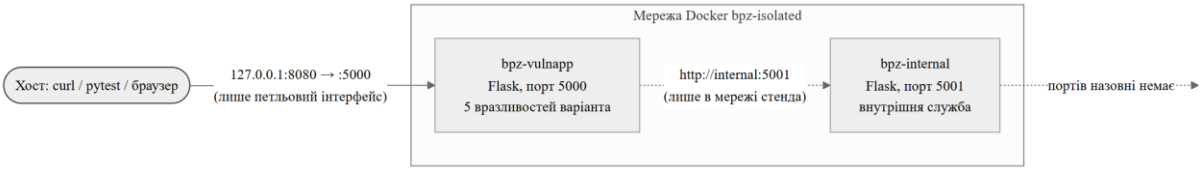


Рисунок 2.1 – Логічна схема ізольованого стенда

Публікацію саме на 127.0.0.1 задає рядок `"127.0.0.1:${VULNAPP_PORT}:5000"` у docker-compose.yml; це унеможливлює доступ до вразливого застосунку ззовні хоста. Регресійні тести виконуються на хості (requests + pytest) проти працюючого стенда.

3 ПІДТВЕРДЖЕННЯ, УСУНЕННЯ ТА РЕГРЕСІЙНИЙ КОНТРОЛЬ

Для кожної з п'яти вразливостей наведено доказ експлуатації (curl-команда й відповідь сервера на 127.0.0.1), фрагмент коду «до/після» та регресійний тест. Усі дані стенда синтетичні.

3.1 B1. SQL-ін’єкція у формі входу (CWE-89 / OWASP A03, /login)

Причина. Запит складається конкатенацією рядків, тому значення login стає частиною коду SQL; `admin' --` коментує перевірку пароля й повертає рядок адміністратора.

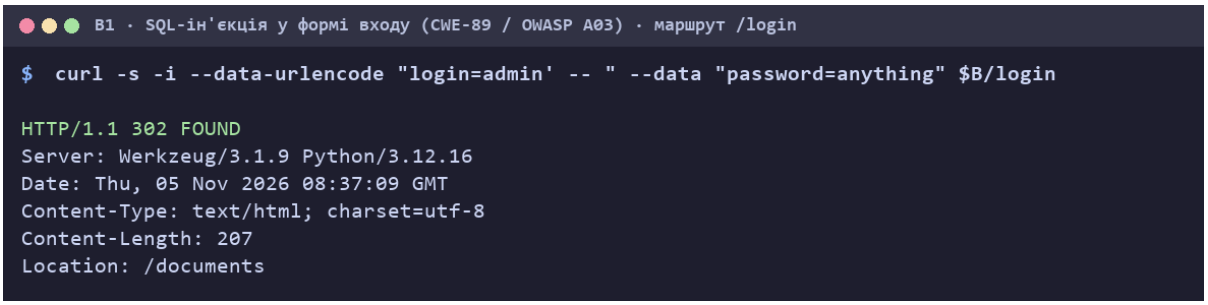


Рисунок 3.1 – Доказ експлуатації B1 на 127.0.0.1 (форма Д1)

Код до виправлення:

```
query = ("SELECT * FROM users WHERE login = '" + login_name + "'"
         " AND password_hash = '" + _weak_hash(password) + "'")
row = db().execute(query).fetchone()
```

Код після виправлення:

```
row = db().execute(
    "SELECT * FROM users WHERE login = ? AND password_hash = ?",
    (login_name, _weak_hash(password)),
).fetchone()
```

Суть виправлення. Параметризований запит: значення передано окремо від тексту SQL, тому їх не можна інтерпретувати як код. Штатний вхід правильними даними не змінюється.

Регресійний тест:

```
def test_v1_sql_injection_login_bypass():
    s = requests.Session()
    r = s.post(f"{BASE}/login", data={"login": "admin' -- ", "password": "будь-що"},
               allow_redirects=False, timeout=TIMEOUT)
    # На виправленому коді ін'єкція не автентифікує: немає переходу на /documents
    assert not (r.status_code == 302 and r.headers.get("Location",
"").endswith("/documents")), \
        "SQL-ін'єкція admin' -- автентифікувала без пароля"
    assert not _is_authenticated(s), "ін'єкція створила дійсну сесію"
    # Класичне ' OR 1=1 -- теж не має проходити
    s2 = requests.Session()
    s2.post(f"{BASE}/login", data={"login": "' OR 1=1 -- ", "password": "x"},
timeout=TIMEOUT)
    assert not _is_authenticated(s2), "ін'єкція ' OR 1=1 -- автентифікувала"
    # Штатний вхід правильними даними має й далі працювати
    assert _is_authenticated(_session(*USER)), "легальний вхід зламано виправленням"
```

3.2 В3. Відображений XSS у пошуку (CWE-79 / OWASP A03, /search)

Причина. Шаблон виводить рядок запиту фільтром `|safe`, який вимикає автоекранування Jinja2, тож `<script>` потрапляє у відповідь як виконуваний HTML.



Рисунок 3.2 – Доказ експлуатації В3 на 127.0.0.1 (форма Д1)

Код до виправлення:

```
{# search.html #}
<p>Результати за запитом: {{ q|safe }}</p>
```

Код після виправлення:

```
{# search.html — без |safe: Jinja2 екранує автоматично #}
<p>Результати за запитом: {{ q }}</p>
```

Суть виправлення. Прибрано фільтр `|safe`; Jinja2 автоматично екранує спецсимволи, і запит виводиться як текст. Пошук за звичайними словами працює без змін.

Регресійний тест:

```
def test_v3_reflected_xss_search():
    payload = "<script>alert(1)</script>"
    r = requests.get(f"{BASE}/search", params={"q": payload}, timeout=TIMEOUT)
    assert payload not in r.text, "рядок запиту виводиться без екранування (відображений
XSS)"
    assert "&lt;script&gt;alert(1)&lt;/script&gt;" in r.text, "немає екранованого вигляду
запиту"
```

3.3 В4. CSRF у зміні профілю (CWE-352 / OWASP A01, /profile)

Причина. Маршрут змінює стан (профіль) за POST без синхронізованого токена й не перевіряє походження запиту, тому стороння сторінка може надіслати форму від імені жертви.

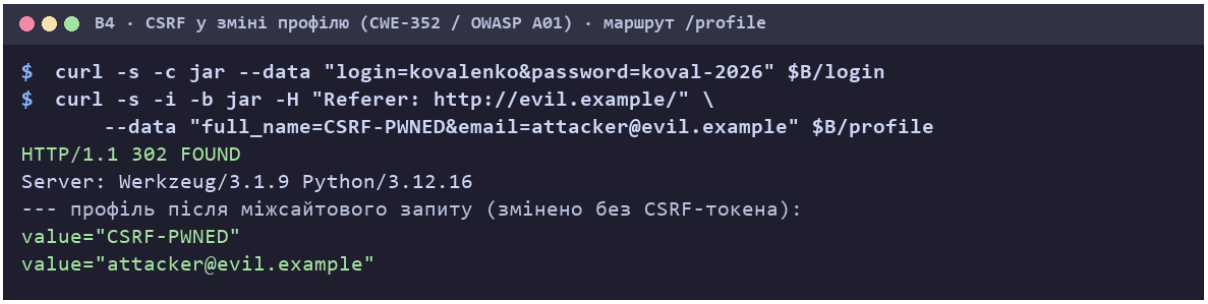


Рисунок 3.3 – Доказ експлуатації B4 на 127.0.0.1 (форма Д1)

Код до виправлення:

```
# POST /profile — зміна стану без перевірки токена
db().execute("UPDATE users SET full_name = ?, email = ? WHERE id = ?",
             (request.form.get("full_name"), request.form.get("email"), user["id"]))
```

Код після виправлення:

```
if not _valid_csrf(request.form.get("csrf_token", "")):
    return "Недійсний CSRF-токен", 403
# форма несе прихований токен сесії; render_template(..., csrf_token=_csrf_token())
```

Суть виправлення. Додано синхронізований токен: `_csrf_token()` кладе випадковий токен у сесію й у приховане поле форми, а POST приймається лише за збігу (`hmac.compare_digest`). Легітимна форма несе токен і працює.

Регресійний тест:

```
def test_v4_csrf_profile_update():
    s = _session(*USER)
    before = s.get(f"{BASE}/profile", timeout=TIMEOUT).text
    # Міжсайтовий запит: стороння сторінка, без CSRF-токена
    r = s.post(f"{BASE}/profile",
               data={"full_name": "CSRF-PWNED", "email": "attacker@evil.example"},
               headers={"Referer": "http://evil.example/attack.html"},
               allow_redirects=False, timeout=TIMEOUT)
    assert r.status_code in (400, 403), \
        f"зміну профілю без CSRF-токена прийнято (код {r.status_code})"
    after = s.get(f"{BASE}/profile", timeout=TIMEOUT).text
    assert "attacker@evil.example" not in after and before == after, \
        "профіль змінено міжсайтовим запитом (CSRF)"
```

3.4 B5. IDOR у перегляді документів (CWE-639 / OWASP A01, /document/<id>)

Причина. Маршрут віддає документ за `id` без перевірки власника, тому будь-який автентифікований користувач читає чужі конфіденційні документи.

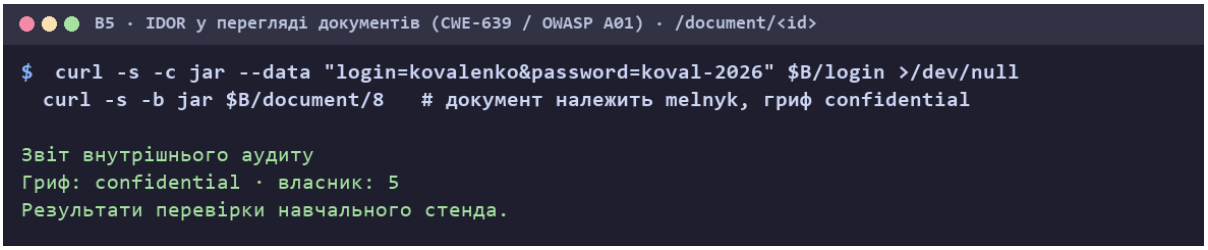


Рисунок 3.4 – Доказ експлуатації B5 на 127.0.0.1 (форма Д1)

Код до виправлення:

```
row = db().execute("SELECT * FROM documents WHERE id = ?", (doc_id,)).fetchone()
if row is None:
    return "Документ не знайдено", 404
# власник не перевіряється
```

Код після виправлення:

```
if row["owner_id"] != user["id"] and user["role"] not in ("admin", "auditor"):
    return "Документ не знайдено", 404
```

Суть виправлення. Додано перевірку `owner_id == uid` (ролі admin/auditor зберігають нагляд); чужий документ повертає 404. Власні документи доступні, як і раніше.

Регресійний тест:

```
def test_v5_idor_document():
    s = _session(*USER)                                    # kovalenko, id=2
    r = s.get(f"{BASE}/document/8", allow_redirects=False, timeout=TIMEOUT)  # власник — 
melnyk
    assert r.status_code in (403, 404), \
        f"доступ до чужого документа дозволено (код {r.status_code}, IDOR)"
    assert "Результати перевірки" not in r.text, "вміст чужого документа розкрито"
    # Власний документ має лишатися доступним
    own = s.get(f"{BASE}/document/2", timeout=TIMEOUT)     # документ kovalenko
    assert own.status_code == 200 and "Кошторис" in own.text, "власний документ став 
недоступним"
```

3.5 B10. Відсутність перевірки прав (CWE-862 / OWASP A01, /admin/users)

Причина. Адмінмаршрут не перевіряє ні автентифікації, ні ролі, тому список усіх користувачів доступний навіть анонімному відвідувачеві.

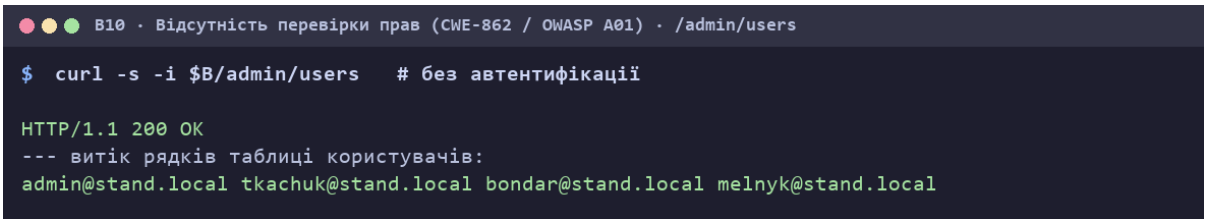


Рисунок 3.5 – Доказ експлуатації B10 на 127.0.0.1 (форма Д1)

Код до виправлення:

```
# /admin/users — без перевірки прав
rows = db().execute("SELECT id, login, full_name, email, role FROM
users").fetchall()
return render_template("admin_users.html", user=current_user(), users=rows)
```

Код після виправлення:

```
user = current_user()
if user is None:
    return redirect(url_for("login"))
if user["role"] != "admin":
    return "Недостатньо прав", 403
```

Суть виправлення. Додано явну перевірку: анонім — перенаправлення на вхід, не-адміністратор — 403, адміністратор зберігає доступ.

Регресійний тест:

```
def test_v10_admin_requires_authorization():
    # Анонім
    r = requests.get(f"{BASE}/admin/users", allow_redirects=False, timeout=TIMEOUT)
    assert r.status_code in (302, 403), f"адмінсторінка доступна анонімно (код
{r.status_code})"
    assert "melnyk@stand.local" not in r.text, "список користувачів витік анонімно"
    # Звичайний користувач (не admin) теж не має доступу
    s = _session(*USER)
    r2 = s.get(f"{BASE}/admin/users", allow_redirects=False, timeout=TIMEOUT)
    assert r2.status_code in (302, 403), "звичайний користувач отримав адмінсторінку"
    # Адміністратор доступ зберігає
    adm = _session("admin", "Adm1n-stand-2026")
    r3 = adm.get(f"{BASE}/admin/users", timeout=TIMEOUT)
    assert r3.status_code == 200 and "melnyk" in r3.text, "адміністратор втратив доступ"
```

4 РЕЗУЛЬТАТИ ТЕСТУВАННЯ

Файл test_regression.py запущено проти стенда двічі. На вразливому коді всі п’ять тестів червоні (кожен відтворює експлуатацію й фіксує її наявність):

```
FAILED test_regression.py::test_v1_sql_injection_login_bypass - AssertionErro...
FAILED test_regression.py::test_v3_reflected_xss_search - AssertionError: ряд...
FAILED test_regression.py::test_v4_csrf_profile_update - AssertionError: змін...
FAILED test_regression.py::test_v5_idor_document - AssertionError: доступ до ...
FAILED test_regression.py::test_v10_admin_requires_authorization - AssertionE...
============================== 5 failed in 0.81s ===============================
```

Після внесення п’яти виправлень і перебудови образу (`docker compose up -d --build vulnapp`) ті самі тести зелені:

```
PASSED test_regression.py::test_v1_sql_injection_login_bypass
PASSED test_regression.py::test_v3_reflected_xss_search
PASSED test_regression.py::test_v4_csrf_profile_update
PASSED test_regression.py::test_v5_idor_document
PASSED test_regression.py::test_v10_admin_requires_authorization
5 passed in 0.86s
```

Повторну перевірку команд кроку 3 на виправленому стенді зведено на рис. 4.1: жодна з п’яти вразливостей не відтворюється, а штатна робота (легальний вхід, доступ до власного документа) збережена.

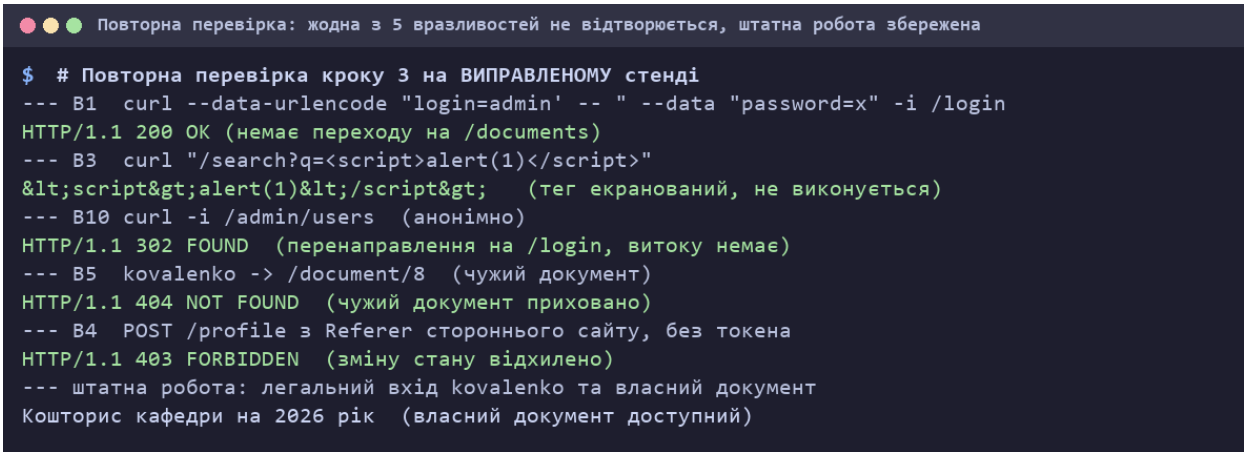


Рисунок 4.1 – Повторна перевірка на виправленому стенді

5 АНАЛІЗ РЕЗУЛЬТАТІВ

5.1 Категорії, причини та наслідки

Три з п’яти вразливостей належать до OWASP A01 (порушення розмежування доступу): IDOR (B5), CSRF (B4) і відсутність перевірки прав (B10); дві — до A03 (ін’єкції): SQL-ін’єкція (B1) і відображений XSS (B3). Наслідки різні: B1 дає обхід автентифікації (вхід як admin без пароля), B10 — розкриття даних усіх користувачів анонімно, B5 — доступ до чужого конфіденційного документа, B4 — зміну стану від імені жертви, B3 — виконання довільного скрипта в браузері жертви.

5.2 Чому навантаження спрацьовує

B1: `admin' --` закриває рядковий літерал і коментує решту запиту, тож пароль не перевіряється. B3: `|safe` вимикає екранування, і `<script>` інтерпретується браузером. B4: відсутній невідомий сторонній стороні токен, тому запит із чужого сайту проходить. B5: рішення спирається лише на `id` з URL. B10: немає жодної перевірки суб’єкта.

5.3 Мінімальність виправлень

Кожне виправлення прибирає причину, не змінюючи інтерфейсу маршруту: параметризація (B1) і прибирання `|safe` (B3) не впливають на штатні запити; перевірка власника (B5) і прав (B10) пропускає легітимних користувачів; CSRF-токен (B4) додається прозоро до форми. Це підтверджено тим, що після виправлень легальний вхід, пошук, перегляд власних документів і доступ адміністратора працюють (рис. 4.1).

5.4 Що перевіряє регресійний тест

Кожен тест відтворює те саме навантаження, що й доказ, і стверджує відсутність вразливості: B1 — ін'єкція не створює сесії; B3 — у відповіді немає сирого `<script>`, є екранований; B4 — зміну без токена відхилено (403) і профіль не змінився; B5 — чужий документ дає 404, власний — 200; B10 — анонім і не-адміністратор отримують 302/403, адміністратор — 200. На вразливому коді кожне твердження хибне (тест червоний), на виправленому — істинне (зелений). Крім негативних, тести містять позитивні перевірки штатної роботи, тож виправлення не можна «пройти» простим блокуванням маршруту.

5.5 Можливість обходу виправлень

B1: Параметризація закриває всі ін'єкційні навантаження, а не лише `--`; додатково варто зберігати паролі повільною KDF із сіллю (ЛР3) замість MD5. B3: Самого екранування достатньо для цього виводу; у масштабі застосунку його доповнюють політикою CSP, щоб обмежити виконання вбудованих скриптів навіть у разі помилки. B4: Токен не обходиться підміною Referer; додатково cookie сесії варто позначити `SameSite=Lax`, що блокує міжсайтову відправку cookie. B5: Обхід неможливий, доки рішення спирається на `owner_id` із БД, а не на дані клієнта; за складнішої моделі доступу перевірку виносять у централізований диспетчер (пор. ЛР2). B10: Перевірку слід дублювати на кожному адмінмаршруті; системно це вирішує декоратор-ґард або централізований диспетчер доступу.

5.6 Порівняння за впливом

Найбільший потенційний збиток дає B1 (SQL-ін’єкція у /login): вона не потребує автентифікації й одразу надає права адміністратора, поєднуючи обхід автентифікації з повним доступом — орієнтовно CVSS 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H). Далі — B10 (анонімне розкриття даних усіх користувачів, C:H, але без зміни стану) і B5 (доступ до чужих конфіденційних документів, потрібна автентифікація). B4 потребує взаємодії жертви (UI:R), а B3 впливає насамперед на окремого користувача, тож їх вплив нижчий.

6 ВИСНОВКИ

1) Розгорнуто ізольований стенд (дві служби мережі bpz-isolated), застосунок доступний лише на 127.0.0.1:8080; у коді за маркерами визначено п’ять вразливостей варіанта (B1, B3, B4, B5, B10).
2) Експлуатацію кожної підтверджено на 127.0.0.1 у формі Д1 (curl-команда й відповідь сервера): обхід автентифікації через SQL-ін’єкцію, виконання `<script>` у пошуку, зміна профілю міжсайтовим запитом, читання чужого документа, анонімне розкриття списку користувачів.
3) Усі п’ять дефектів усунено мінімальними змінами (параметризація, прибирання `|safe`, CSRF-токен, перевірка власника, перевірка прав); штатна робота маршрутів збережена, що підтверджено позитивними перевірками.
4) Для кожної вразливості створено регресійний тест; на вразливому коді — 5 failed, на виправленому — 5 passed. Повторна перевірка команд кроку 3 показала, що жодна вразливість не відтворюється.
5) Зміни стосуються лише п’яти маршрутів варіанта й не зачіпають реалізацій інших варіантів у тому самому застосунку.

7 ВІДОМОСТІ ПРО ВИКОРИСТАННЯ ГЕНЕРАТИВНИХ ІНСТРУМЕНТІВ

Під час виконання роботи як допоміжний засіб використано генеративну модель Claude (Anthropic): для підтвердження експлуатації, підготовки виправлень коду, регресійних тестів і оформлення звіту. Усі дії виконано локально проти стенда на 127.0.0.1; результати перевірено запуском тестів; рішення здобувач перевірив і може пояснити під час захисту.

ПЕРЕЛІК ПОСИЛАНЬ

1) OWASP Top 10:2021. The Ten Most Critical Web Application Security Risks [Електронний ресурс] / OWASP Foundation. – Режим доступу: https://owasp.org/Top10/
2) CWE – Common Weakness Enumeration [Електронний ресурс] / MITRE. – Режим доступу: https://cwe.mitre.org/
3) OWASP Cheat Sheet Series [Електронний ресурс] / OWASP Foundation. – Режим доступу: https://cheatsheetseries.owasp.org/
4) OWASP Application Security Verification Standard (ASVS) 4.0.3 [Електронний ресурс] / OWASP Foundation.
5) Common Vulnerability Scoring System v3.1: Specification Document / FIRST [Електронний ресурс]. – Режим доступу: https://www.first.org/cvss/
6) Безпека програмного забезпечення. Частина 2. Лабораторний практикум / В. В. Соколовський. – Київ : КПІ ім. Ігоря Сікорського, 2026.

ДОДАТОК А

Репозиторій і артефакти роботи

Репозиторій: https://github.com/ip34is/7-sem (папка SoS/lab5)

Коміт, що відповідає зданій роботі:

`004b82b8e38973f0aedb4b83aff5d35c98c4fd01`

Стан роботи на цьому коміті: https://github.com/ip34is/7-sem/tree/004b82b8e38973f0aedb4b83aff5d35c98c4fd01/SoS/lab5

Артефакти: lab05_vulnapp/app.py та templates/ (виправлені), lab05_vulnapp/test_regression.py, docker-compose.yml; у каталозі звіт_лр5 — proof_*.png (докази Д1), scheme.png, pytest_before.txt, pytest_after.txt, compose_ps.txt. Усі облікові записи й дані стенда синтетичні.

Відтворення (з папки SoS/lab5, потрібен Docker):

```
cp .env.example .env
docker compose up -d --build        # застосунок на 127.0.0.1:8080
cd lab05_vulnapp
PYTHONPATH=. python -m pytest -q test_regression.py   # 5 passed
docker compose down -v
```